

Remote WhatsApp-to-Agent Command Loop

Eigenständiges technisches Deep-Dive-Dossier

Remote WhatsApp-to-Agent Command Loop

Ich wollte von unterwegs mit meinem Handy Arbeit an meinen Laptop übergeben koennen, ohne daraus eine gefaehrliche direkte Shell zu machen. Die WhatsApp-Nachricht sollte erst verstanden, in einen Arbeitsauftrag übersetzt, ausgeführt, bewertet und dann sauber zurückgemeldet werden.

- Das stärkste Signal ist nicht der Chatkanal, sondern die sichere Übersetzung von Alltagssprache in einen geprüften Arbeitsauftrag: Eingang, Interpretation, Ausführung, Bewertung und erst danach Rückmeldung.
- Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.

Was ich mir dabei gedacht habe

Was ich mir dabei gedacht habe, war einfach, aber technisch anspruchsvoll: Ich wollte nicht am Laptop sitzen müssen und trotzdem sinnvoll Arbeit an ihn übergeben können, ohne mein Handy zu einer blinden Remote-Shell zu machen. Der interessante Teil war die Vertrauensgrenze. Eine Nachricht musste erst zum Arbeitsauftrag werden, ausgeführt und danach bewertet werden, bevor sie zu mir zurückkommt.

Was ich gebaut habe

- Ich habe Fernzugriff als Autoritäts- und Sicherheitsproblem behandelt, nicht nur als Komfortfunktion.
- Ich habe die Front-Agenten-Schicht von der tool-reichen Ausführungsschicht getrennt.
- Ich habe mobile Sprache/Chat in strukturierte Arbeit mit Erfolgskriterien übersetzt.

Mechanik im Detail

- Der wertvolle Teil ist nicht WhatsApp selbst. WhatsApp ist nur der reibungsarme Eingang. Der harte Teil ist die Übersetzung: eine lockere Nachricht vom Handy wird zu einem strukturierten Arbeitsauftrag mit Ziel, Komplexität, Modellhinweis und Erfolgskriterien.
- Das Dual-Brain-Muster trennt die Rolle, die versteht und bewertet, von der Rolle, die mit Tools ausführt. Dadurch wird eine Handy-Nachricht nicht zu einem ungeprüften Direktbefehl.
- Die Antwort kommt bewusst erst nach der Bewertung zurück. Das ist näher an Supervisor/Executor als an einem normalen Chatbot.

Architektur

- WhatsApp oder Audio kommt in der OpenClaw-Schicht an.
- Ein Watcher normalisiert Ereignisse, dedupliziert Nachrichten und behandelt Medien.
- MiguelAgent macht daraus Ziel, Komplexität, Arbeitsauftrag, Erfolgskriterien und Modell-Hinweis.
- Die Ausführungsseite nutzt Miguel/EXOVIUM-Tools und Modellrouting.
- Eine Qualitätsschleife prüft, ob das Ergebnis wirklich fertig ist.
- Erst danach wird die Antwort wieder an WhatsApp geschickt.

Evidenzcluster

- Workflow-Notizen zu WhatsApp-Event-Erkennung, Audio-Transkription, strukturierten Arbeitsaufträgen, Modellhinweisen und Quality-Loop-Schwellen.
- Pipeline-Notizen zur Kette WhatsApp -> Gateway -> Watcher -> MiguelAgent -> Ausführung/Bewertung/Formatierung -> Antwort.
- Python-Watcher- und MiguelAgent-Module mit Event-Parsing, Medienhandling, Deduplication, Arbeitsauftragsausführung, Evaluator-Loop und Antwortformatierung.
- Entscheidungsnotizen zu schneller Bestätigung, Smart Routing und erfolgreichen End-to-End-Tests.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Das stärkste Signal ist nicht der Chatkanal, sondern die sichere Übersetzung von Alltagssprache in einen geprüften Arbeitsauftrag: Eingang, Interpretation, Ausführung, Bewertung und erst danach Rückmeldung.	Workflow-Notizen zu WhatsApp-Event-Erkennung, Audio-Transkription, strukturierten Arbeitsaufträgen, Modellhinweisen und Quality-Loop-Schwellen.; Pipeline-Notizen zur Kette WhatsApp -> Gateway -> Watcher -> MiguelAgent -> Ausführung/Bewertung/Formatierung -> Antwort.	Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	WhatsApp oder Audio kommt in der OpenClaw-Schicht an.; Ein Watcher normalisiert Ereignisse, dedupliziert Nachrichten und behandelt Medien.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Das stärkste Signal ist nicht der Chatkanal, sondern die sichere Übersetzung von Alltagssprache in einen geprüften Arbeitsauftrag: Eingang, Interpretation, Ausführung, Bewertung und erst danach Rückmeldung.

- Autoritätsgrenzen: Der Remote-Kanal ist bequem, aber das System behandelt ihn als riskante Kontrollfläche.
- Operatives Denken: schnelle Bestätigung, Deduplication, Medienhandling und finale Rückmeldung sind getrennte Fehlerpunkte.
- Evaluator-Loop-Intuition: Das System führt nicht nur aus, sondern prüft, ob das Ergebnis den ursprünglichen Auftrag erfüllt.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Remote WhatsApp-to-Agent Command Loop.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.

- Schnell bestaetigen, langsam und kontrolliert arbeiten, erst nach Bewertung antworten.
- Remote-Steuerung braucht Deduplikation, Unterbrechbarkeit, Fortschritt und Audit-Spuren.
- Private Kanal-IDs und Zugangsdaten gehoeren nie in ein öffentliches Portfolio.

Nächste Härtung / nächster Projektschritt

- Öffentlichen Demo-Modus mit synthetischen Daten bauen.
- Befehle nach Risiko und Bestaetigungsbedarf klassifizieren.
- Tool-Aufrufe, Modellwahl und Bewertung als sichere Audit-Events speichern.
- Einen öffentlichen synthetischen Demo-Flow bauen, in dem ein Mock-WhatsApp-Event zu einem Arbeitsauftrag wird, ein Fake-Executor läuft und ein Evaluator freigibt oder Nacharbeit verlangt.
- Jeden Übergang loggen: Eingang, Normalisierung, Work Order, Tool Plan, Executor Result, Evaluator Result und Antwort.
- Permission-Klassen für Read-only, Write, Shell, Netzwerk und irreversible Aktionen hinzufügen.